Deep Learning para visión de computadoras

Unidad 2. Detección de objetos, segmentación y secuencias visuales **2.1** Detección de objetos **2.4** Análisis de movimiento en video



INSTRUCCIONES

Antes de empezar: `Entorno de ejecución → Cambiar tipo de entorno → GPU (T4)`.

📝 Las celdas marcadas con

✍️ **RESPONDE** se contestan *en el propio notebook*

🔧 **AJUSTA** piden modificar parámetros y volver a correr.

0. Preparación del entorno

```
!pip install -q ultralytics
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.9/46.9 kB 2.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 24.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 96.1/96.1 kB 3.0 MB/s eta 0:00:00
```

```
import os, time, base64, subprocess, shutil, warnings
from collections import Counter, defaultdict
import cv2, numpy as np, pandas as pd
import matplotlib.pyplot as plt
import torch, ultralytics
from ultralytics import YOLO, solutions
from ultralytics.utils.downloads import safe_download
from IPython.display import HTML, display
import logging
from ultralytics.utils import LOGGER
warnings.filterwarnings("ignore")
LOGGER.setLevel(logging.WARNING)
pd.set_option("display.width", 160)

DEVICE = 0 if torch.cuda.is_available() else "cpu"
print("Ultralytics", ultralytics.__version__, "| dispositivo:", torch.cuda.get_device_name(0) if DEVICE == 0 else "C

ASSETS = os.path.join(os.path.dirname(ultralytics.__file__), "assets")
IMG_BUS, IMG_ZIDANE = os.path.join(ASSETS, "bus.jpg"), os.path.join(ASSETS, "zidane.jpg")

URL = "https://github.com/ultralytics/assets/releases/download/v0.0.0/"
VIDEO_CALLE = "solutions_ci_demo.mp4"        # personas caminando (≈270 KB)
VIDEO_POSE  = "solution_ci_pose_demo.mp4"    # persona haciendo ejercicio (≈58 KB)
for f in [VIDEO_CALLE, VIDEO_POSE]:
    if not os.path.exists(f):
        safe_download(URL + f)

model = YOLO("yolo11n.pt")
pose_model = YOLO("yolo11n-pose.pt")
```

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.
Ultralytics 8.4.174 | dispositivo: Tesla T4
Downloading https://ultralytics.com/assets/solutions_ci_demo.mp4 to 'solutions_ci_demo.mp4': 100% ━━━━━━━━━━━━ 264.6K
Downloading https://ultralytics.com/assets/solution_ci_pose_demo.mp4 to 'solution_ci_pose_demo.mp4': 100% ━━━━━━━━━━
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% ━━━━━━━━━━
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n-pose.pt to 'yolo11n-pose.pt': 100%
```

```
def mostrar(img_bgr, titulo="", ancho=9):
    h, w = img_bgr.shape[:2]
    plt.figure(figsize=(ancho, ancho * h / w))
    plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)); plt.axis("off"); plt.title(titulo); plt.show()

def mostrar_video(ruta, ancho=640):
    salida = ruta.replace(".mp4", "_web.mp4")
    subprocess.run(["ffmpeg", "-y", "-loglevel", "error", "-i", ruta, "-vcodec", "libx264",
                    "-pix_fmt", "yuv420p", salida], check=False)
    if not os.path.exists(salida):
        print("No se pudo convertir; descarga el archivo:", ruta); return
    b64 = base64.b64encode(open(salida, "rb").read()).decode()
    display(HTML(f'<video width="{ancho}" controls loop><source src="data:video/mp4; base64,{b64}" type="video/mp4"><

def info_video(ruta):
    cap = cv2.VideoCapture(ruta)
    d = dict(ancho=int(cap.get(3)), alto=int(cap.get(4)), fps=cap.get(5), frames=int(cap.get(7)))
    cap.release(); return d

print(info_video(VIDEO_CALLE)); print(info_video(VIDEO_POSE))
```

```
{'ancho': 640, 'alto': 360, 'fps': 30.0, 'frames': 62}
{'ancho': 640, 'alto': 360, 'fps': 30.0, 'frames': 60}
```

(Opcional) Usar tu propio video. Grábalo con el celular: **máximo 10 s**, horizontal, y súbelo con la celda de abajo. Después cambia `VIDEO_CALLE = "tu_video.mp4"`.

```
# OPCIONAL — subir video propio (descomenta)
from google.colab import files
subido = files.upload()
#VIDEO_CALLE = "tu_video.mp4"
VIDEO_CALLE = list(subido.keys())[0]; print(info_video(VIDEO_CALLE))
```

```
Elegir archivos Sin archivos seleccionados Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving tu_video.mp4 to tu_video (1).mp4
{'ancho': 1920, 'alto': 1080, 'fps': 25.0, 'frames': 350}
```

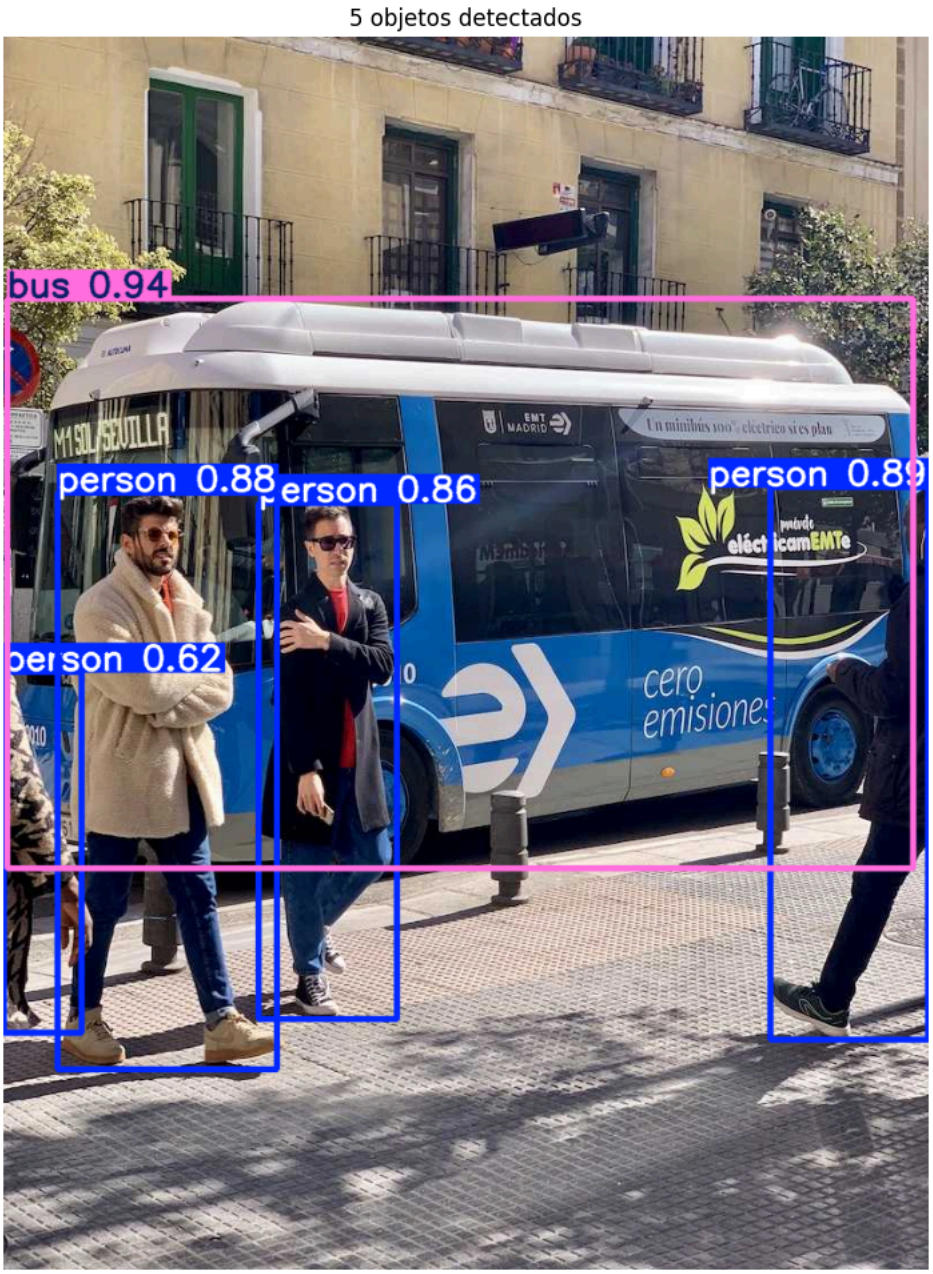
PARTE A — Detección en imagen

Recuerda la mesa redonda: YOLO resuelve la detección como un **problema dual** — *localización* (¿dónde?, la caja) + *clasificación* (¿qué?, la clase) — en una sola pasada de la red. Aquí vamos a abrir el objeto `results` y trabajar directamente con los números de cada caja.

A1. Primera detección y anatomía de `results.boxes`

```
res = model(IMG_BUS, conf=0.25, iou=0.7, imgsz=640, device=DEVICE, verbose=False)[0]
mostrar(res.plot(), f"{len(res.boxes)} objetos detectados")

b = res.boxes
print("xyxy  (x1,y1,x2,y2) →", b.xyxy[0].tolist())
print("xywh  (cx,cy,w,h)   →", b.xywh[0].tolist())
print("xywhn (normalizado)  →", [round(v, 3) for v in b.xywhn[0].tolist()])
print("conf →", round(float(b.conf[0]), 3), "| cls →", int(b.cls[0]), "=", res.names[int(b.cls[0])])
print("Tiempos (ms):", {k: round(v, 1) for k, v in res.speed.items()})
```




```
xyxy  (x1,y1,x2,y2) → [3.8326663970947266, 229.36419677734375, 796.194580078125, 728.4122314453125]
xywh  (cx,cy,w,h)   → [400.01361083984375, 478.8882141113281, 792.3619384765625, 499.04803466796875]
xywhn (normalizado)  → [0.494, 0.443, 0.978, 0.462]
conf → 0.94 | cls → 5 = bus
Tiempos (ms): {'preprocess': 2.4, 'inference': 10.3, 'postprocess': 1.6}
```

```
def cajas_a_df(res):
    b = res.boxes
    if len(b) == 0:
        return pd.DataFrame(columns=["clase", "conf", "x1", "y1", "x2", "y2", "cx", "cy", "w", "h", "area"])
    xyxy, xywh = b.xyxy.cpu().numpy(), b.xywh.cpu().numpy()
    df = pd.DataFrame(xyxy, columns=["x1", "y1", "x2", "y2"])
    df[["cx", "cy", "w", "h"]] = xywh
    df["area"] = df.w * df.h
    df = df.round(1)
    df.insert(0, "conf", b.conf.cpu().numpy().round(3))
    df.insert(0, "clase", [res.names[int(c)] for c in b.cls])
    return df

df_bus = cajas_a_df(res)
df_bus
```

	clase	conf	x1	y1	x2	y2	cx	cy	w	h	area
0	bus	0.940	3.800000	229.399994	796.200012	728.400024	400.000000	478.899994	792.400024	499.000000	395426.593750
1	person	0.888	671.000000	394.799988	809.799988	878.700012	740.400024	636.799988	138.800003	483.899994	67158.796875
2	person	0.878	47.400002	399.600006	239.300003	904.200012	143.399994	651.900024	191.899994	504.600006	96836.398438
3	person	0.856	223.100006	408.700012	344.500000	860.400024	283.799988	634.599976	121.400002	451.700012	54846.000000
4	person	0.622	0.000000	556.099976	68.900002	872.400024	34.500000	714.200012	68.900002	316.299988	21781.000000

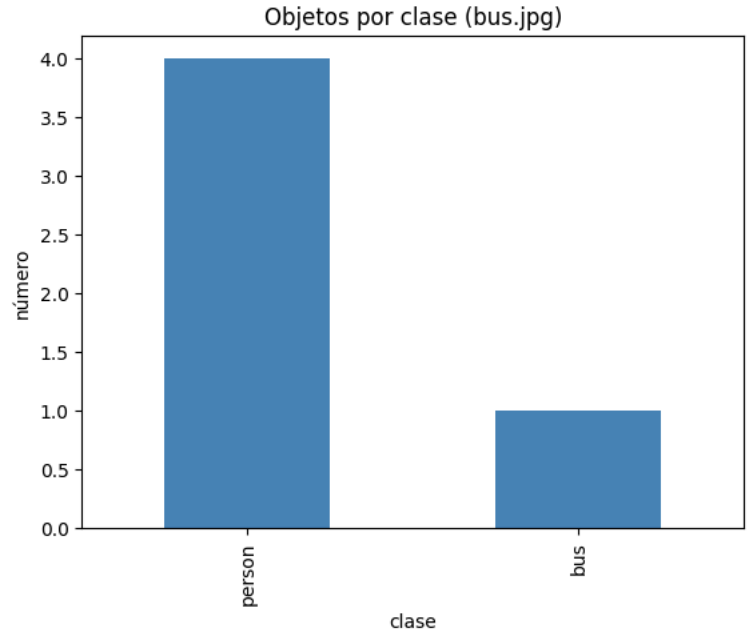
A2. ¿Cuántos objetos hay? Conteo por clase

```
def contar(res):
    return Counter(res.names[int(c)] for c in res.boxes.cls)

for nombre, img in [("bus.jpg", IMG_BUS), ("zidane.jpg", IMG_ZIDANE)]:
    r = model(img, device=DEVICE, verbose=False)[0]
    print(f"{nombre:11s} total={len(r.boxes):2d} →", dict(contar(r)))

conteo = df_bus.clase.value_counts()
conteo.plot.bar(color="steelblue", title="Objetos por clase (bus.jpg)"); plt.ylabel("número"); plt.show()
```

```
bus.jpg     total= 5 → {'bus': 1, 'person': 4}
zidane.jpg  total= 3 → {'person': 2, 'tie': 1}
```



A3. ¿De qué color es cada objeto?

YOLO no sabe de colores: **eso lo calculamos nosotros** con la caja. Idea: recortar la caja, quedarnos con la zona central (para no medir el fondo), pasar a HSV y tomar el *tono* (H) dominante de los píxeles con suficiente saturación. Si casi no hay saturación, el objeto es blanco, gris o negro según su brillo (V).

```
# Rangos de tono en OpenCV (H va de 0 a 179)
RANGOS_H = [(0, 8, "rojo"), (8, 22, "naranja"), (22, 34, "amarillo"), (34, 85, "verde"),
            (85, 100, "cian"), (100, 130, "azul"), (130, 160, "morado"), (160, 180, "rojo")]
BGR_COLOR = {"rojo": (40, 40, 220), "naranja": (0, 140, 255), "amarillo": (0, 220, 240), "verde": (60, 180, 60),
             "cian": (220, 200, 0), "azul": (220, 80, 30), "morado": (160, 50, 140),
             "blanco": (245, 245, 245), "gris": (140, 140, 140), "negro": (20, 20, 20)}

def color_dominante(img_bgr, caja, recorte_central=0.5, sat_min=60, val_min=50):
    x1, y1, x2, y2 = map(int, caja)
    w, h = x2 - x1, y2 - y1
    m = (1 - recorte_central) / 2                    # margen a quitar por lado
    roi = img_bgr[int(y1 + m * h):int(y2 - m * h), int(x1 + m * w):int(x2 - m * w)]
    if roi.size == 0:
```

```
        return "desconocido", 0.0
    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV).reshape(-1, 3)
    cromaticos = hsv[(hsv[:, 1] >= sat_min) & (hsv[:, 2] >= val_min)]
    if len(cromaticos) < 0.15 * len(hsv):           # casi sin color → acromático
        v = np.median(hsv[:, 2])
        return ("negro" if v < 60 else "gris" if v < 170 else "blanco"), 1 - len(cromaticos) / len(hsv)
    hist = np.bincount(cromaticos[:, 0], minlength=180)
    h_dom = int(np.argmax(np.convolve(hist, np.ones(5), "same")))  # pico suavizado
    nombre = next(n for a, b_, n in RANGOS_H if a <= h_dom < b_)
    return nombre, len(cromaticos) / len(hsv)

def agregar_colores(df, img_bgr, **kw):
    df = df.copy()
    res_col = [color_dominante(img_bgr, r[["x1", "y1", "x2", "y2"]], **kw) for _, r in df.iterrows()]
    df["color"] = [c for c, _ in res_col]; df["pureza"] = [round(p, 2) for _, p in res_col]
    return df

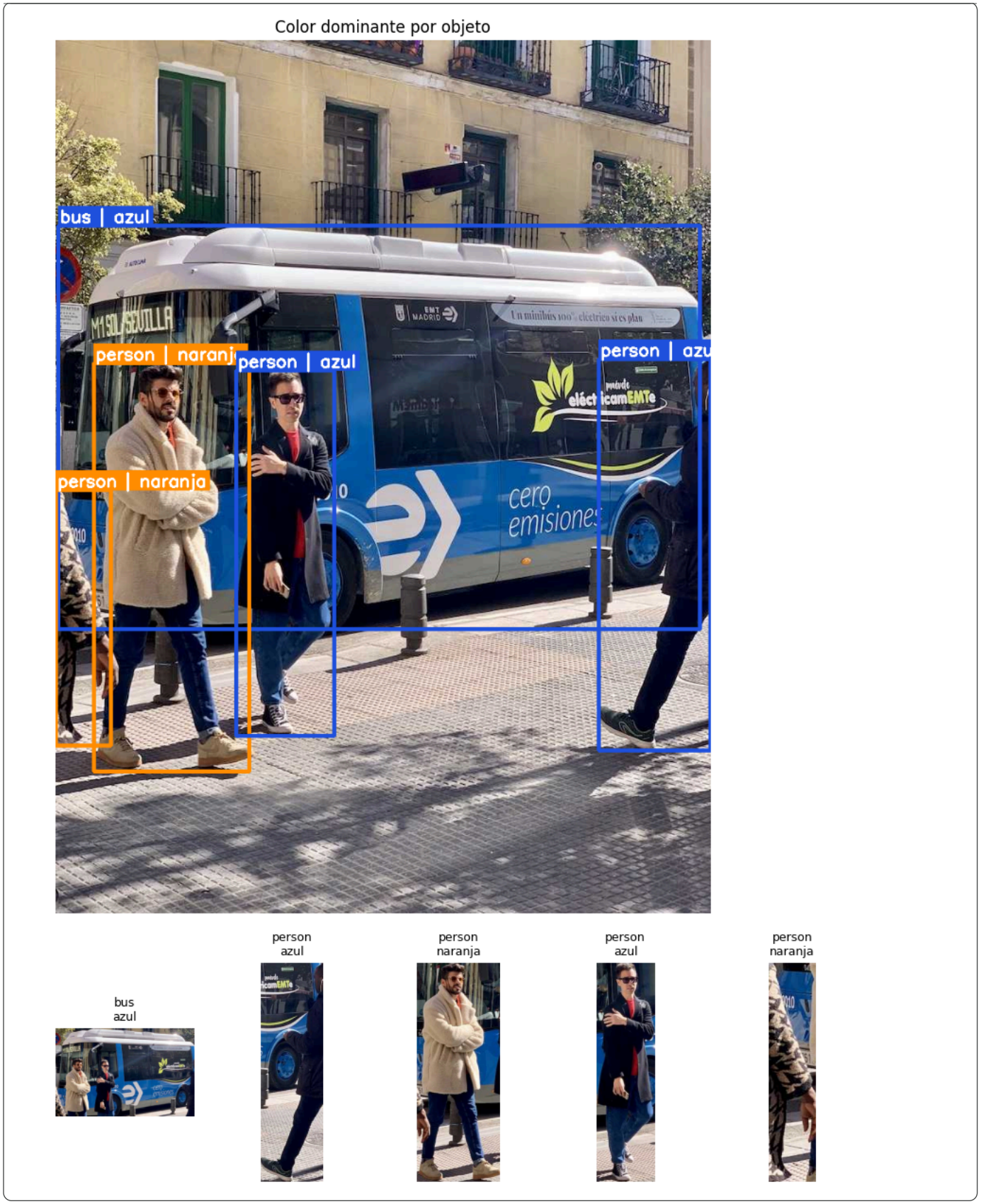
img_bus = cv2.imread(IMG_BUS)
df_bus = agregar_colores(df_bus, img_bus)
df_bus[["clase", "conf", "color", "pureza", "area"]]
```

	clase	conf	color	pureza	area
0	bus	0.940	azul	0.60	395426.593750
1	person	0.888	azul	0.35	67158.796875
2	person	0.878	naranja	0.47	96836.398438
3	person	0.856	azul	0.28	54846.000000
4	person	0.622	naranja	0.29	21781.000000

```
# Dibujamos nuestras PROPIAS cajas: borde del color detectado + etiqueta
def dibujar_colores(img_bgr, df):
    out = img_bgr.copy()
    for _, r in df.iterrows():
        p1, p2 = (int(r.x1), int(r.y1)), (int(r.x2), int(r.y2))
        col = BGR_COLOR.get(r.color, (255, 255, 255))
        cv2.rectangle(out, p1, p2, col, 4)
        txt = f"{r.clase} | {r.color}"
        (tw, th), _ = cv2.getTextSize(txt, cv2.FONT_HERSHEY_SIMPLEX, 0.7, 2)
        cv2.rectangle(out, (p1[0], p1[1] - th - 8), (p1[0] + tw + 4, p1[1]), col, -1)
        cv2.putText(out, txt, (p1[0] + 2, p1[1] - 4), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
                    (0, 0, 0) if r.color in ("blanco", "amarillo", "cian") else (255, 255, 255), 2)
    return out

mostrar(dibujar_colores(img_bus, df_bus), "Color dominante por objeto")

# Recortes de cada objeto, para verificar a ojo
fig, ax = plt.subplots(1, len(df_bus), figsize=(2.2 * len(df_bus), 3))
for a, (_, r) in zip(np.atleast_1d(ax), df_bus.iterrows()):
    crop = img_bus[int(r.y1):int(r.y2), int(r.x1):int(r.x2)]
    a.imshow(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)); a.set_title(f"{r.clase}\n{r.color}", fontsize=9); a.axis("off")
plt.show()
```

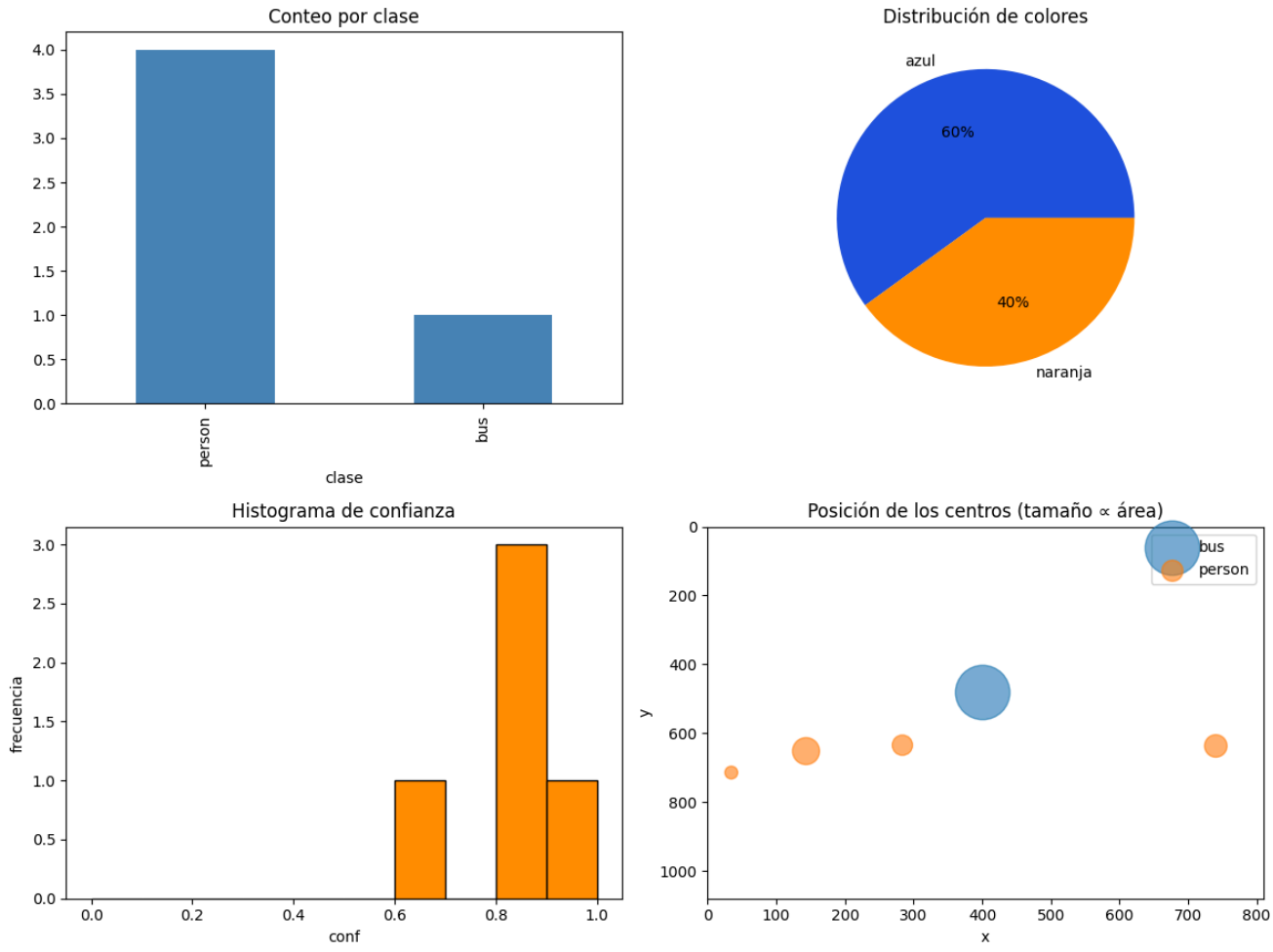


✍️ **RESPONDE A3**

1. ¿En qué objetos acertó el color y en cuáles falló? Explica *por qué* falló (fondo dentro de la caja, ropa de varios colores, iluminación, sombras...).
2. 🔧 **AJUSTA** `recorte_central` (0.3, 0.5, 0.8) y `sat_min` (30, 60, 100). ¿Qué combinación funcionó mejor y por qué?
3. Propón una mejora (p. ej. K-means con 3 grupos dentro de la caja, o usar la máscara de `yolo11n-seg`). No es obligatorio implementarla, pero suma.

A4. Gráficas de los objetos detectados

```
fig, ax = plt.subplots(2, 2, figsize=(12, 9))
df_bus.clase.value_counts().plot.bar(ax=ax[0, 0], color="steelblue", title="Conteo por clase")
df_bus.color.value_counts().plot.pie(ax=ax[0, 1], autopct="%d%%", title="Distribución de colores",
                                     colors=[np.array(BGR_COLOR.get(c, (128,)*3))[::-1] / 255 for c in df_bus.color.
ax[0, 1].set_ylabel("")
ax[1, 0].hist(df_bus.conf, bins=10, range=(0, 1), color="darkorange", edgecolor="k")
ax[1, 0].set(title="Histograma de confianza", xlabel="conf", ylabel="frecuencia")
H, W = img_bus.shape[:2]
for cl, g in df_bus.groupby("clase"):
    ax[1, 1].scatter(g.cx, g.cy, s=g.area / 300, alpha=.6, label=cl)
ax[1, 1].set(xlim=(0, W), ylim=(H, 0), title="Posición de los centros (tamaño ∝ área)", xlabel="x", ylabel="y")
ax[1, 1].legend(); plt.tight_layout(); plt.show()
```



A5. IoU y NMS con tus propias manos

En la mesa redonda vimos que **IoU** (*Intersection over Union*) mide cuánto se traslapan dos cajas, y que **NMS** usa el IoU para eliminar cajas duplicadas. Implementa la función y compárala con la de Ultralytics.

```
def iou(a, b):
    # a, b = [x1, y1, x2, y2]
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
```

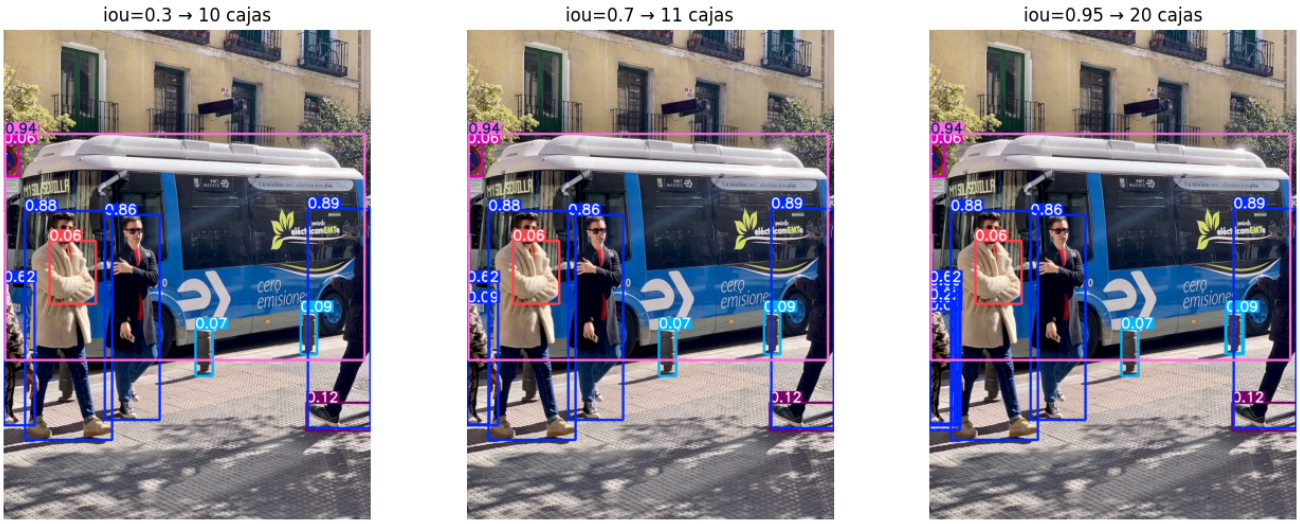
```
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = (a[2] - a[0]) * (a[3] - a[1]); area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter + 1e-9)

from ultralytics.utils.metrics import box_iou
cajas = res.boxes.xyxy.cpu()
mi_matriz = np.array([[iou(a.tolist(), b.tolist()) for b in cajas] for a in cajas])
print("¿Coincide con Ultralytics?", np.allclose(mi_matriz, box_iou(cajas, cajas).numpy(), atol=1e-4))
pd.DataFrame(mi_matriz.round(2), index=df_bus.clase, columns=df_bus.clase)
```

```
¿Coincide con Ultralytics? True
```

clase	bus	person	person	person	person
clase					
bus	1.00	0.1	0.15	0.09	0.03
person	0.10	1.0	0.00	0.00	0.00
person	0.15	0.0	1.00	0.05	0.06
person	0.09	0.0	0.05	1.00	0.00
person	0.03	0.0	0.06	0.00	1.00

```
# 🔧 AJUSTA: efecto del umbral IoU de NMS (conf muy bajo para provocar duplicados)
fig, ax = plt.subplots(1, 3, figsize=(16, 6))
for a, u in zip(ax, [0.3, 0.7, 0.95]):
    r = model(IMG_BUS, conf=0.05, iou=u, device=DEVICE, verbose=False)[0]
    a.imshow(cv2.cvtColor(r.plot(labels=False), cv2.COLOR_BGR2RGB)); a.axis("off")
    a.set_title(f"iou={u} → {len(r.boxes)} cajas")
plt.show()
```



✍️ **RESPONDE A5**

1. ¿Por qué con `iou=0.95` aparecen más cajas? Explícalo con el algoritmo de NMS.
2. ¿Qué pasaría con `iou=0.3` en una foto de una multitud muy apretada?
3. Prueba `agnostic_nms=True`. ¿Qué cambia y en qué caso real te conviene?

A6. Barrido de parámetros de inferencia

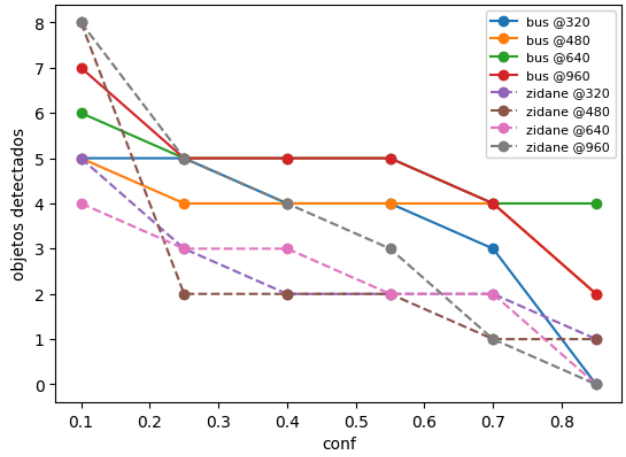
Los parámetros que **todos** los P deberán justificar en la Parte C:

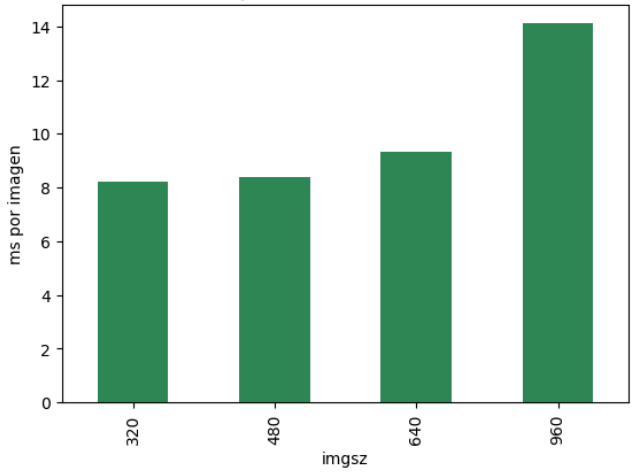
Parámetro	Qué controla	Efecto típico al subirlo
`conf`	Umbral mínimo de confianza	Menos falsos positivos, más objetos perdidos
`iou`	Umbral de NMS	Más cajas duplicadas sobreviven

Parámetro	Qué controla	Efecto típico al subirlo
`imgsz`	Resolución de entrada	Detecta objetos más pequeños, pero es más lento
`max_det`	Máximo de detecciones por imagen	Útil en escenas muy pobladas
`classes`	Filtra clases (p. ej. `[0]` = solo personas)	Menos ruido y menos cómputo en post-proceso

```
filas = []
for imgsz in [320, 480, 640, 960]:
    for conf in [0.1, 0.25, 0.4, 0.55, 0.7, 0.85]:
        for img_n, img in [("bus", IMG_BUS), ("zidane", IMG_ZIDANE)]:
            r = model(img, conf=conf, imgsz=imgsz, device=DEVICE, verbose=False)[0]
            filas.append(dict(img=img_n, imgsz=imgsz, conf=conf, n=len(r.boxes),
                              personas=int((r.boxes.cls == 0).sum()), ms=r.speed["inference"]))
barrido = pd.DataFrame(filas)

fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
for (img_n, imgsz), g in barrido.groupby(["img", "imgsz"]):
    ax[0].plot(g.conf, g.n, marker="o", ls="-" if img_n == "bus" else "--", label=f"{img_n} @{imgsz}")
ax[0].set(xlabel="conf", ylabel="objetos detectados", title="Detecciones vs umbral de confianza"); ax[0].legend(font
barrido.groupby("imgsz").ms.median().plot.bar(ax=ax[1], color="seagreen", title="Tiempo de inferencia (mediana)")
ax[1].set_ylabel("ms por imagen"); plt.show()
barrido.pivot_table(index="conf", columns=["img", "imgsz"], values="n")
```

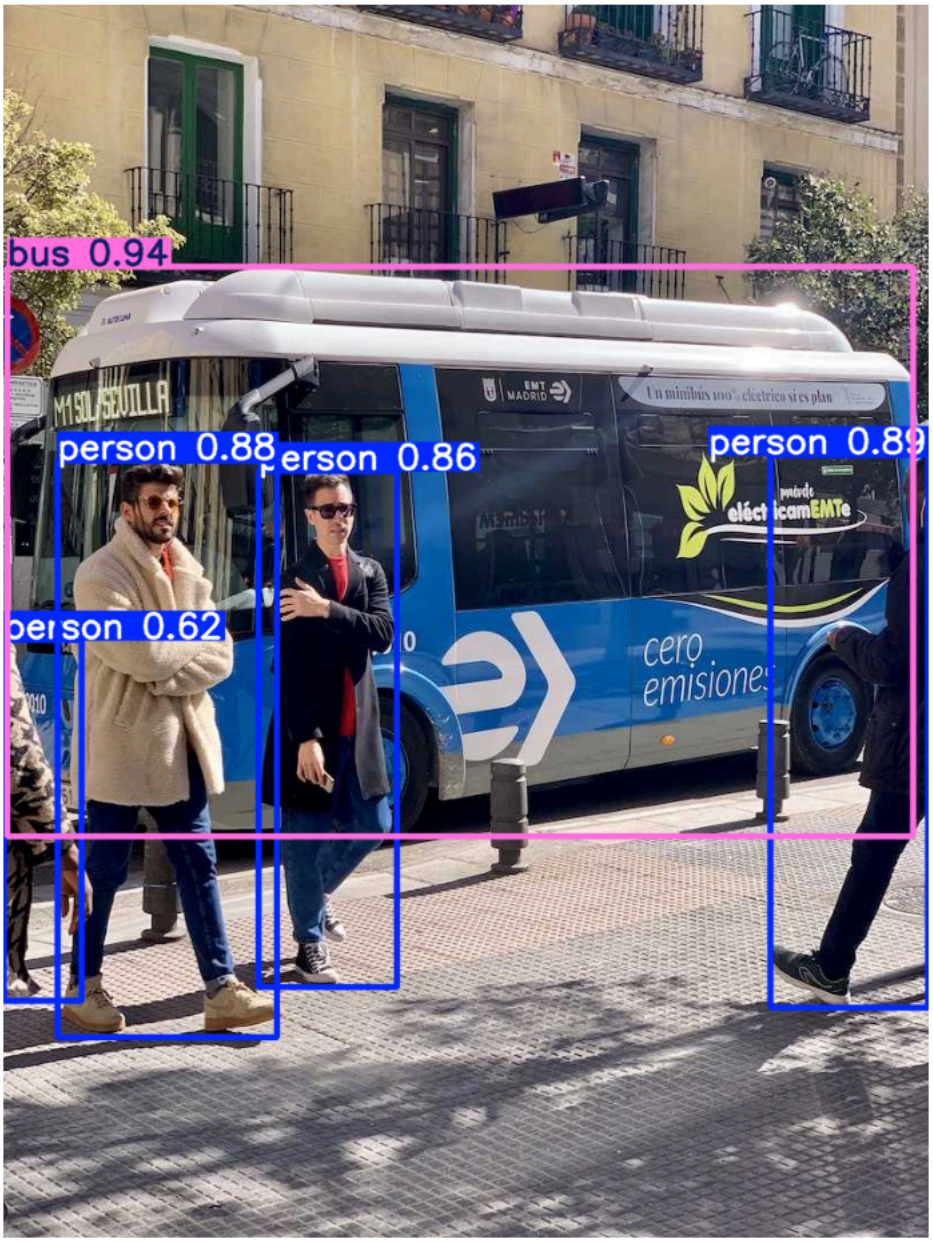




img	bus				zidane			
imgsz	320	480	640	960	320	480	640	960
conf								
0.10	5.0	5.0	6.0	7.0	5.0	8.0	4.0	8.0
0.25	5.0	4.0	5.0	5.0	3.0	2.0	3.0	5.0
0.40	4.0	4.0	5.0	5.0	2.0	2.0	3.0	4.0
0.55	4.0	4.0	5.0	5.0	2.0	2.0	2.0	3.0
0.70	3.0	4.0	4.0	4.0	2.0	1.0	2.0	1.0
0.85	0.0	2.0	4.0	2.0	1.0	1.0	0.0	0.0

```
# 🔧 AJUSTA: elige TU configuración y justifícala
MI_CONF, MI_IOU, MI_IMGSZ, MIS_CLASES = 0.4, 0.6, 640, None   # ej. MIS_CLASES=[0, 5] (persona, autobús)
r = model(IMG_BUS, conf=MI_CONF, iou=MI_IOU, imgsz=MI_IMGSZ, classes=MIS_CLASES, max_det=50, device=DEVICE, verbose=
mostrar(r.plot(), f"Mi configuración: {dict(contar(r))}")
```

Mi configuración: {'bus': 1, 'person': 4}



A7. Métricas de detección: Precision, Recall, mAP50 y mAP50-95

Ahora medimos contra **etiquetas reales** (ground truth) usando `coco8` (8 imágenes, descarga < 1 MB). Recuerda:

- **mAP50**: una predicción cuenta como correcta si su IoU con la caja real es ≥ 0.5.
- **mAP50-95**: promedio de mAP con umbrales de IoU de 0.50 a 0.95 — exige cajas mucho más precisas.

```
met = []
for imgsz in [320, 640]:
    for conf in [0.001, 0.25]:
        v = model.val(data="coco8.yaml", imgsz=imgsz, conf=conf, device=DEVICE, plots=False, verbose=False)
        met.append(dict(imgsz=imgsz, conf=conf, P=v.box.mp, R=v.box.mr, mAP50=v.box.map50, mAP50_95=v.box.map))
df_met = pd.DataFrame(met).round(3); df_met
```

```
WARNING ⚠ Dataset 'coco8.yaml' images not found, missing path '/content/datasets/coco8/images/val'
Downloading https://ultralytics.com/assets/coco8.zip to '/content/datasets/coco8.zip': 100% ━━━━━━━━━━━━ 432.8KB 16.9
Unzipping /content/datasets/coco8.zip to /content/datasets/coco8...: 100% ━━━━━━━━━━━━ 25/25 3.8Kfiles/s 0.0s
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% ━━━━━━━━━━━━ 755.
```

	imgsz	conf	P	R	mAP50	mAP50_95
0	320	0.001	0.644	0.567	0.684	0.40
1	320	0.250	0.350	0.300	0.284	0.23
2	640	0.001	0.570	0.850	0.846	0.63
3	640	0.250	0.694	0.650	0.643	0.47

```
# Métricas por clase (última validación)
pd.DataFrame({"clase": [v.names[i] for i in v.box.ap_class_index],
              "AP50": v.box.ap50.round(3), "AP50-95": v.box.ap.round(3)})
```

	clase	AP50	AP50-95
0	person	0.375	0.208
1	dog	0.995	0.696
2	horse	0.995	0.671
3	elephant	0.495	0.248
4	umbrella	0.995	0.995
5	potted plant	0.000	0.000

Ajuste fino de N épocas

Igual que en el laboratorio anterior: el modelo **ya está entrenado**, solo lo afinamos 3 épocas sobre `coco8` y comparamos.

```
LOGGER.setLevel(logging.INFO)      # queremos ver la tabla de pérdidas por época
modelo_ft = YOLO("yolo11n.pt")
modelo_ft.train(data="coco8.yaml", epochs=3, imgsz=320, batch=8, device=DEVICE,
                plots=False, verbose=False, name="ft_coco8", exist_ok=True)
LOGGER.setLevel(logging.WARNING)
v_ft = modelo_ft.val(data="coco8.yaml", imgsz=320, device=DEVICE, plots=False, verbose=False)
v_or = YOLO("yolo11n.pt").val(data="coco8.yaml", imgsz=320, device=DEVICE, plots=False, verbose=False)
pd.DataFrame({"original": [v_or.box.mp, v_or.box.mr, v_or.box.map50, v_or.box.map],
              "3 épocas": [v_ft.box.mp, v_ft.box.mr, v_ft.box.map50, v_ft.box.map]},
             index=["Precision", "Recall", "mAP50", "mAP50-95"]).round(3)
```

```
Ultralytics 8.4.174 🚀 Python-3.13.15 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=8, bgr=0.0, b
```

✍️ **RESPONDE A6–A7**

1. Con tu barrido: ¿qué `conf` elegirías para (a) un sistema de **seguridad** que no puede perder personas y (b) un **conteo de inventario** donde un falso positivo cuesta dinero? Argumenta con la curva.
2. ¿Cuánto más lento es `imgsz=960` que `imgsz=320`? ¿Cuándo vale la pena pagar ese costo?
3. ¿Por qué `mAP50-95` siempre es menor que `mAP50`?
4. ¿Por qué `model.val()` usa por defecto `conf=0.001` y no `0.25`? (Pista: la curva Precision-Recall necesita *todas* las predicciones.)
5. ¿Las 3 épocas mejoraron o empeoraron el mAP? Con solo 4 imágenes de validación, ¿qué tan confiable es esa conclusión?

PARTE B — Seguimiento y análisis de movimiento en video

En video, detectar no basta: necesitamos saber que la persona del frame 10 **es la misma** del frame 11. Eso lo hace el **tracker**, que asigna un `id` a cada objeto combinando:

- la predicción de movimiento (filtro de Kalman) de dónde *debería* estar cada `id`, y
- el emparejamiento (por IoU y/o apariencia) con las cajas nuevas que da YOLO.

Ultralytics trae varios: **ByteTrack**, **BoT-SORT**, **OC-SORT**, **TrackTrack** (por defecto en esta versión), FastTrack y Deep OC-SORT.

B1. Seguimiento y tabla de trayectorias

```
def seguir(video, tracker="bytetrack.yaml", conf=0.25, iou=0.7, imgsz=640, classes=None, guardar=None):
    # Ejecuta model.track frame por frame y devuelve un DataFrame con cada caja seguida
    filas, escritor, t0 = [], None, time.time()
    gen = model.track(video, stream=True, persist=True, tracker=tracker, conf=conf, iou=iou,
                      imgsz=imgsz, classes=classes, device=DEVICE, verbose=False)
    for f, r in enumerate(gen):
        if guardar:
            frame = r.plot()
            if escritor is None:
                escritor = cv2.VideoWriter(guardar, cv2.VideoWriter_fourcc(*"mp4v"), info_video(video)["fps"],
                                           (frame.shape[1], frame.shape[0]))
            escritor.write(frame)
        if r.boxes.id is None:
            continue
        for tid, c, cf, (cx, cy, w, h), xyxy in zip(r.boxes.id.int().tolist(), r.boxes.cls.int().tolist(),
                                                    r.boxes.conf.tolist(), r.boxes.xywh.tolist(), r.boxes.xyxy.tolis
            filas.append(dict(frame=f, id=tid, clase=r.names[c], conf=cf, cx=cx, cy=cy, w=w, h=h,
                              x1=xyxy[0], y1=xyxy[1], x2=xyxy[2], y2=xyxy[3]))
    if escritor: escritor.release()
    df = pd.DataFrame(filas); df.attrs["seg"] = time.time() - t0
    return df

tracks = seguir(VIDEO_CALLE, guardar="tracking_bytetrack.mp4")
print(f"Filas: {len(tracks)} | IDs únicos: {tracks.id.nunique()} | tiempo: {tracks.attrs['seg']:.1f}s")
mostrar_video("tracking_bytetrack.mp4")
tracks.head()
```

```
3 epochs completed in 0.001 hours.
Optimizer stripped from /content/runs/detect/ft_coco8/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/ft_coco8/weights/best.pt, 5.5MB

Validating /content/runs/detect/ft_coco8/weights/best.pt...
Ultralytics 8.4.174 🚀 Python-3.13.15 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 4.8it
                   all          4         17      0.664      0.567      0.711      0.416
Speed: 0.1ms preprocess, 45.1ms inference, 0.0ms loss, 1.5ms postprocess per image
```

	original	3 épocas
Precision	0.644	0.666
Recall	0.567	0.567
mAP50	0.684	0.704
mAP50-95	0.400	0.415

```
WARNING ⚠ requirements: Restart runtime or rerun command for updates to take effect

Filas: 3725 | IDs únicos: 135 | tiempo: 18.9s
```

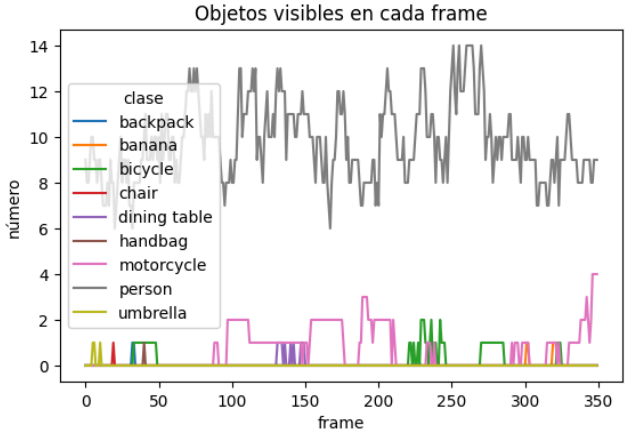
0:00 / 0:14

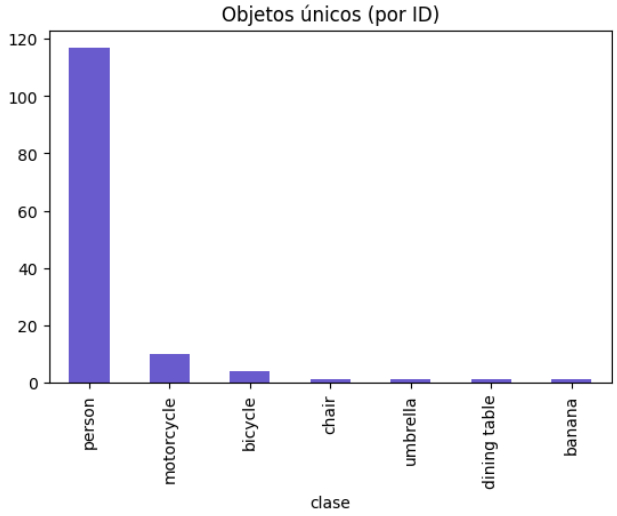
	frame	id	clase	conf	cx	cy	w	h	x1	y1	x2	y2
0	0	1	person	0.853746	1104.617920	911.014526	423.055969	330.467285	893.089905	745.780884	1316.145874	1076.248169
1	0	2	person	0.792805	400.803650	846.950745	216.772034	466.098511	292.417633	613.901489	509.189667	1080.000000
2	0	3	person	0.789197	81.978859	761.314148	163.325668	621.812622	0.316023	450.407837	163.641693	1072.220459
3	0	4	person	0.742702	1455.820557	903.364746	182.873779	350.426514	1364.383667	728.151489	1547.257446	1078.578003
4	0	5	person	0.520898	783.002930	734.528931	180.401733	205.205811	692.802063	631.926025	873.203796	837.131836

B2. ¿Cuántos objetos hay *realmente*? Conteo por frame vs. conteo por ID

```
por_frame = tracks.groupby(["frame", "clase"]).size().unstack(fill_value=0)
resumen = tracks.groupby("id").agg(clase=("clase", "first"), frames=("frame", "count"),
                                   conf_media=("conf", "mean")).round(2)

fig, ax = plt.subplots(1, 2, figsize=(14, 4))
por_frame.plot(ax=ax[0], title="Objetos visibles en cada frame"); ax[0].set_ylabel("número")
resumen.clase.value_counts().plot.bar(ax=ax[1], color="slateblue", title="Objetos únicos (por ID)")
plt.show()
print("Máximo simultáneo:", por_frame.sum(axis=1).max(), "| Objetos únicos:", len(resumen))
resumen
```






```
Máximo simultáneo: 15 | Objetos únicos: 135
```

id	clase	frames	conf_media
1	person	149	0.70
2	person	57	0.57
3	person	20	0.78
4	person	94	0.67
5	person	9	0.50
...	...	...	...
621	person	5	0.83
624	motorcycle	6	0.34
629	person	4	0.38
630	motorcycle	5	0.31
636	motorcycle	4	0.50

135 rows × 3 columns

```
# Color de cada objeto seguido: votación por mayoría a lo largo de los frames
cap, colores_por_id = cv2.VideoCapture(VIDEO_CALLE), defaultdict(list)
for f in range(int(tracks.frame.max()) + 1):
    ok, frame = cap.read()
    if not ok: break
    for _, r in tracks[tracks.frame == f].iterrows():
        colores_por_id[r.id].append(color_dominante(frame, (r.x1, r.y1, r.x2, r.y2))[0])
cap.release()
resumen["color"] = [Counter(colores_por_id[i]).most_common(1)[0][0] for i in resumen.index]
resumen["estabilidad_color"] = [round(Counter(colores_por_id[i]).most_common(1)[0][1] / len(colores_por_id[i]), 2)
                                for i in resumen.index]
resumen
```

id	clase	frames	conf_media	color	estabilidad_color
1	person	149	0.70	azul	0.38
2	person	57	0.57	azul	0.81
3	person	20	0.78	azul	1.00
4	person	94	0.67	azul	0.28
5	person	9	0.50	verde	0.89
...	...	...	...	...	...
621	person	5	0.83	azul	1.00
624	motorcycle	6	0.34	negro	1.00
629	person	4	0.38	rojo	1.00
630	motorcycle	5	0.31	negro	1.00
636	motorcycle	4	0.50	negro	1.00

135 rows × 5 columns

```
# Trayectorias sobre el último frame + velocidad aparente (px/frame)
cap = cv2.VideoCapture(VIDEO_CALLE); cap.set(cv2.CAP_PROP_POS_FRAMES, int(tracks.frame.max())); _, fondo = cap.read()
plt.figure(figsize=(11, 6)); plt.imshow(cv2.cvtColor(fondo, cv2.COLOR_BGR2RGB))
for tid, g in tracks.groupby("id"):
    plt.plot(g.cx, g.cy, lw=2, label=f"id {tid}"); plt.scatter(g.cx.iloc[-1], g.cy.iloc[-1], s=30)
plt.legend(fontsize=7, ncol=2); plt.axis("off"); plt.title("Trayectorias de los centros"); plt.show()

tracks = tracks.sort_values(["id", "frame"])
tracks["vel_px"] = np.hypot(tracks.groupby("id").cx.diff(), tracks.groupby("id").cy.diff())
tracks["dir_x"] = np.sign(tracks.groupby("id").cx.diff())
vel = tracks.groupby("id").agg(vel_media=("vel_px", "mean"), sentido=("dir_x", lambda s: "→" if s.mean() > 0 else "←"
vel
```



B3. Comparación de trackers

Un buen tracker produce **pocos IDs** (no "inventa" personas nuevas cuando alguien se tapa) y **tracks largos**. Un número de IDs mucho mayor que el número real de personas indica **cambios de identidad (ID switches)**.

⚠️ **Cuidado con la trampa:** *menos IDs no siempre es mejor*. Un tracker con umbrales altos (por ejemplo **TrackTrack**, `track_high_thresh=0.6`), que es el tracker **por defecto** de las Solutions en Ultralytics 8.4) simplemente **ignora** a las personas lejanas con baja confianza. Compara siempre `ids_unicos` contra el *máximo simultáneo* de B2.

```
comp = []
for trk in ["bytetrack.yaml", "botsort.yaml", "ocsort.yaml", "tracktrack.yaml"]:
    model = YOLO("yolo11n.pt")          # modelo nuevo: el tracker no debe heredar IDs del anterior
    t = seguir(VIDEO_CALLE, tracker=trk)
    largo = t.groupby("id").size()
    comp.append(dict(tracker=trk.replace(".yaml", ""), ids_unicos=t.id.nunique(),
                     largo_medio=round(largo.mean(), 1), tracks_cortos=(largo < 5).sum(), segundos=round(t.attrs["se
df_trk = pd.DataFrame(comp); df_trk
```

	id	vel_media	sentido
1	17.79	→	
2	16.05	→	
3	32.26	→	
629	10.59	←	
630	0.37	←	
636	4.99	←	

135 rows × 2 columns

	tracker	ids_unicos	largo_medio	tracks_cortos	segundos
0	bytetrack	135	27.6	34	9.0
1	botsort	125	30.3	26	14.8
2	ocsort	114	33.3	20	9.0
3	tracktrack	43	46.5	4	15.7

```
# 🔧 AJUSTA: crea tu propio tracker modificando sus hiperparámetros
import yaml
ruta = os.path.join(os.path.dirname(ultralytics.__file__), "cfg", "trackers", "bytetrack.yaml")
cfg = yaml.safe_load(open(ruta)); print("Original:", cfg)

cfg.update(track_high_thresh=0.5,   # sube → solo detecciones muy seguras abren/actualizan tracks
           new_track_thresh=0.6,    # sube → cuesta más crear un ID nuevo
           track_buffer=60,         # sube → recuerda más tiempo a quien se ocultó (oclusión)
           match_thresh=0.8)        # umbral de emparejamiento
yaml.safe_dump(cfg, open("mi_tracker.yaml", "w"))

model = YOLO("yolo11n.pt")
t = seguir(VIDEO_CALLE, tracker="mi_tracker.yaml", guardar="tracking_mio.mp4")
print("IDs únicos con mi tracker:", t.id.nunique())
mostrar_video("tracking_mio.mp4")
```

```
Original: {'tracker_type': 'bytetrack', 'track_high_thresh': 0.25, 'track_low_thresh': 0.1, 'new_track_thresh': 0.25,
IDs únicos con mi tracker: 57
```

✍️ **RESPONDE B1–B3**

1. ¿Por qué el *máximo de objetos simultáneos* y el número de *IDs únicos* son distintos? ¿Cuál reportarías a un cliente que pregunta "¿cuántas personas pasaron?"
2. ¿Qué tracker generó menos IDs y tracks más largos? ¿Coincide con lo que ves en el video?
3. TrackTrack generó muchos menos IDs que ByteTrack. ¿Es porque sigue mejor a las personas o porque sigue a *menos* personas? Demuéstralo con `ids_unicos`, el máximo simultáneo y el video.
4. Explica con tus palabras para qué sirve `track_buffer` y qué riesgo tiene subirlo demasiado.
5. ¿El color estimado se mantuvo estable para cada ID? ¿Qué te dice la columna `estabilidad_color`?
6. La velocidad está en **píxeles/frame**. ¿Qué dato extra necesitas para convertirla a km/h?

B4. Flujo óptico: movimiento *sin* detector

El **flujo óptico** (Farnebäck) estima, para cada píxel, un vector de desplazamiento entre dos frames consecutivos. No necesita red neuronal ni clases: solo mide *movimiento*. Es la base de las arquitecturas de dos flujos (*two-stream*) para reconocimiento de acciones.

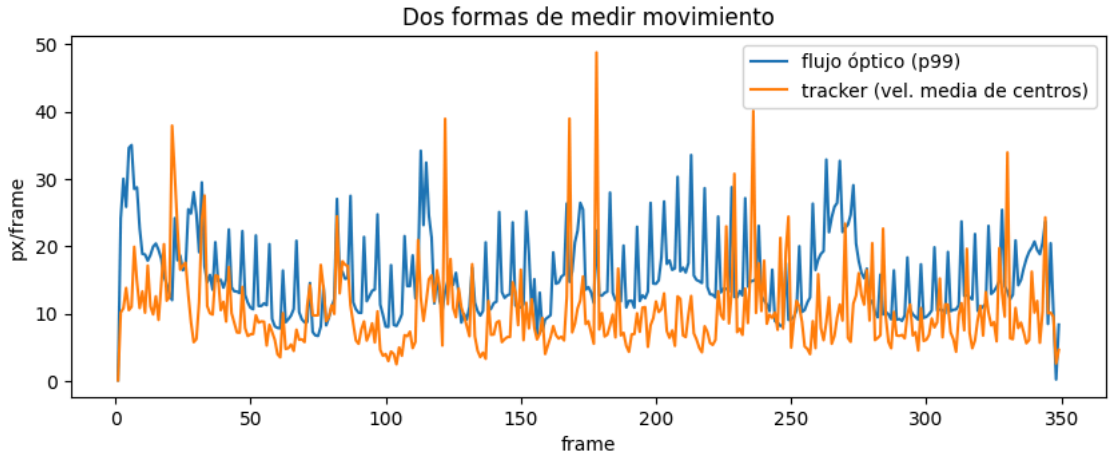
```
cap = cv2.VideoCapture(VIDEO_CALLE); _, prev = cap.read()
prev_g = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)
mag_media, mag_max, ejemplo = [], [], None
while True:
    ok, frame = cap.read()
    if not ok: break
```

```
    g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    flujo = cv2.calcOpticalFlowFarneback(prev_g, g, None, pyr_scale=0.5, levels=3, winsize=15,
                                         iterations=3, poly_n=5, poly_sigma=1.2, flags=0)
    mag, ang = cv2.cartToPolar(flujo[..., 0], flujo[..., 1])
    mag_media.append(mag.mean()); mag_max.append(np.percentile(mag, 99))
    if len(mag_media) == 30:
        hsv = np.zeros_like(frame); hsv[..., 1] = 255
        hsv[..., 0] = ang * 180 / np.pi / 2; hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
        ejemplo = (frame.copy(), cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR))
    prev_g = g
cap.release()

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
ax[0].imshow(cv2.cvtColor(ejemplo[0], cv2.COLOR_BGR2RGB)); ax[0].set_title("Frame 30"); ax[0].axis("off")
ax[1].imshow(cv2.cvtColor(ejemplo[1], cv2.COLOR_BGR2RGB)); ax[1].set_title("Flujo óptico (color = dirección, brillo
ax[2].plot(mag_media, label="media"); ax[2].plot(mag_max, label="percentil 99")
ax[2].set(title="Magnitud del movimiento por frame", xlabel="frame", ylabel="px/frame"); ax[2].legend()
plt.show()
```




```
# Flujo óptico vs tracker: ¿coinciden en cuánto movimiento hay?
vel_tracker = tracks.groupby("frame").vel_px.mean().reindex(range(1, len(mag_max) + 1))
plt.figure(figsize=(10, 3.5))
plt.plot(range(1, len(mag_max) + 1), mag_max, label="flujo óptico (p99)")
plt.plot(vel_tracker.index, vel_tracker.values, label="tracker (vel. media de centros)")
plt.xlabel("frame"); plt.ylabel("px/frame"); plt.legend(); plt.title("Dos formas de medir movimiento"); plt.show()
```



✍ **RESPONDE B4**

1. En la imagen HSV, ¿qué significa que dos personas tengan colores distintos?
2. 🔧 **AJUSTA** `winsize` (5, 15, 31). ¿Qué pasa con el ruido y con el detalle del flujo?
3. Menciona una ventaja del flujo óptico frente al tracker y una ventaja del tracker frente al flujo óptico.
4. **Concepto C3D:** una red 3D convolucional usa filtros de tamaño `3×3×3` (alto × ancho × **tiempo**). Explica qué puede "ver" una C3D que una CNN 2D aplicada frame por frame no puede, y por qué es mucho más costosa. (No la entrenamos: con internet lento, los

pesos pesan cientos de MB.)

PARTE C — Ultralytics Solutions (aplicados)

Parte	Solución	Pregunta de negocio	Video
1	`ObjectCounter`	¿Cuántas personas entran y salen por una línea?	calle
2	`AIGym`	¿Cuántas repeticiones hizo y con qué ángulo?	pose
3	`Heatmap`	¿Por dónde circula más la gente?	calle
4	`SpeedEstimator`	¿A qué velocidad se mueven?	calle
5	`RegionCounter` + `QueueManager`	¿Cuánta gente hay en una zona / en la fila?	calle
6	`TrackZone` + `ObjectBlurrer`	¿Podemos analizar solo una zona y **anonimizar** personas?	calle

> Todas las celdas base usan `tracker="bytetrack.yaml"` para que los resultados sean comparables con la Parte B. Si lo quitan, la Solution usa **TrackTrack** (el valor por defecto) y verán muchos menos objetos: el barrido obligatorio compara ambos.

Obligatorio para TODOS los proyectos

1. Correr la solución con la configuración base y mostrar el video resultante.
2. 🔧 **Barrido de parámetros 2.1**: al menos `conf` (3 valores), `imgsz` (2 valores) y `tracker` (2 valores) usando `barrido_solucion`. La columna `fps_proc` indica cuántos frames por segundo procesó su configuración.
3. Graficar la métrica principal de su solución a lo largo del tiempo.
4. Responder sus preguntas específicas **y** las preguntas comunes C-final.

```
def procesar_video(sol, video, salida=None, max_frames=None, extra=None):
    # Ejecuta una Ultralytics Solution frame por frame. Devuelve (DataFrame de métricas por frame, segundos)
    cap, filas, escritor, t0 = cv2.VideoCapture(video), [], None, time.time()
    fps = cap.get(5) or 30
    f = 0
    nivel = LOGGER.level; LOGGER.setLevel(logging.ERROR)   # evita el aviso "No tracks found" en cada frame
    while cap.isOpened():
        ok, frame = cap.read()
        if not ok or (max_frames and f >= max_frames): break
        r = sol(frame)
        fila = dict(frame=f, t=f / fps, total_tracks=r.total_tracks, in_count=r.in_count, out_count=r.out_count,
                    queue_count=r.queue_count, ms_track=r.speed.get("track", np.nan))
        if extra: fila.update(extra(r))
        filas.append(fila)
        if salida:
            if escritor is None:
                h, w = r.plot_im.shape[:2]
                escritor = cv2.VideoWriter(salida, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))
            escritor.write(r.plot_im)
        f += 1
    cap.release(); LOGGER.setLevel(nivel)
    if escritor: escritor.release()
    df = pd.DataFrame(filas)
    sin = int((df.total_tracks == 0).sum()) if len(df) else 0
    if sin: print(f"⚠ {sin} de {len(df)} frames sin ningún objeto seguido (conf={sol.CFG.get('conf')}, imgsz={sol.C
    return df, time.time() - t0

def barrido_solucion(crear_sol, video, metrica, confs=(0.15, 0.35, 0.6), imgszs=(320, 640),
                     trackers=("bytetrack.yaml", "tracktrack.yaml"), extra=None):
    # crear_sol(conf, imgsz, tracker) debe devolver una solución NUEVA (sin memoria de otra corrida)
    filas = []
    for tr in trackers:
        for sz in imgszs:
            for c in confs:
                df, seg = procesar_video(crear_sol(c, sz, tr), video, extra=extra)
                filas.append(dict(tracker=tr.replace(".yaml", ""), imgsz=sz, conf=c,
                                  resultado=metrica(df), fps_proc=round(len(df) / seg, 1)))
    return pd.DataFrame(filas)

W, H = info_video(VIDEO_CALLE)["ancho"], info_video(VIDEO_CALLE)["alto"]
LINEA = [(20, H // 2), (W - 20, H // 2)]                                    # línea horizontal
ZONA  = [(W // 4, H // 4), (3 * W // 4, H // 4), (3 * W // 4, 3 * H // 4), (W // 4, 3 * H // 4)]  # rectángulo
print("Video", W, "x", H, "| LINEA:", LINEA, "| ZONA:", ZONA)
```

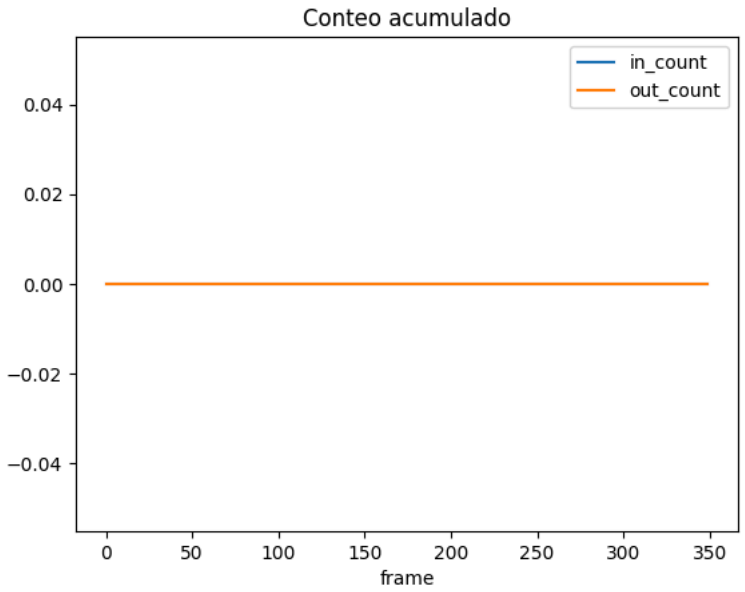
```
Video 1920 x 1080 | LINEA: [(20, 540), (1900, 540)] | ZONA: [(480, 270), (1440, 270), (1440, 810), (480, 810)]
```

Proyecto 1 — `ObjectCounter`: contador de entradas y salidas

Cuenta un objeto cuando su trayectoria **cruza** la línea (o entra/sale del polígono). `classwise_count` desglosa por clase.

```
contador = solutions.ObjectCounter(model="yolo11n.pt", region=LINEA, classes=[0],   # 0 = persona
                                   conf=0.1, iou=0.1, imgsz=640, tracker="bytetrack.yaml",
                                   show=False, verbose=False, device=DEVICE)
df1, seg = procesar_video(contador, VIDEO_CALLE, "eq1_contador.mp4",
                          extra=lambda r: {"por_clase": str(r.classwise_count)})
print(f"ENTRADAS={df1.in_count.iloc[-1]}  SALIDAS={df1.out_count.iloc[-1]}  ({seg:.1f}s)")
mostrar_video("eq1_contador.mp4")
df1[["in_count", "out_count"]].plot(title="Conteo acumulado"); plt.xlabel("frame"); plt.show()
```

```
ENTRADAS=0  SALIDAS=0  (15.5s)
```




```
# 🔧 Barrido obligatorio
b1 = barrido_solucion(lambda c, sz, tr: solutions.ObjectCounter(model="yolo11n.pt", region=LINEA, classes=[0], conf=
                      imgsz=sz, tracker=tr, show=False, verbose=False, device=DEVICE),
                      VIDEO_CALLE, metrica=lambda d: d.in_count.iloc[-1] + d.out_count.iloc[-1])
b1
```

```
⚠️ 25 de 350 frames sin ningún objeto seguido (conf=0.6, imgsz=320)
⚠️ 3 de 350 frames sin ningún objeto seguido (conf=0.35, imgsz=320)
⚠️ 35 de 350 frames sin ningún objeto seguido (conf=0.6, imgsz=320)
⚠️ 1 de 350 frames sin ningún objeto seguido (conf=0.6, imgsz=640)
```

	tracker	imgsz	conf	resultado	fps_proc
0	bytetrack	320	0.15	0	37.4
1	bytetrack	320	0.35	0	44.1
2	bytetrack	320	0.60	0	41.9
3	bytetrack	640	0.15	0	34.5
4	bytetrack	640	0.35	0	37.1
5	bytetrack	640	0.60	0	44.3
6	tracktrack	320	0.15	0	22.4
7	tracktrack	320	0.35	0	22.4
8	tracktrack	320	0.60	0	23.2
9	tracktrack	640	0.15	0	21.1
10	tracktrack	640	0.35	0	21.2
11	tracktrack	640	0.60	0	21.2

✍️ **Proyecto 1:** (a) Cambia la línea a vertical y luego a un polígono (`region=ZONA`). ¿Cuál cuenta mejor en este video y por qué? (b) Cuenta a mano las personas que cruzan. ¿Qué configuración del barrido se acercó más? (c) Si un ID cambia al cruzar la línea, ¿se cuenta doble? Busca evidencia.

Proyecto 2 — `AIGym`: contador de repeticiones por ángulo

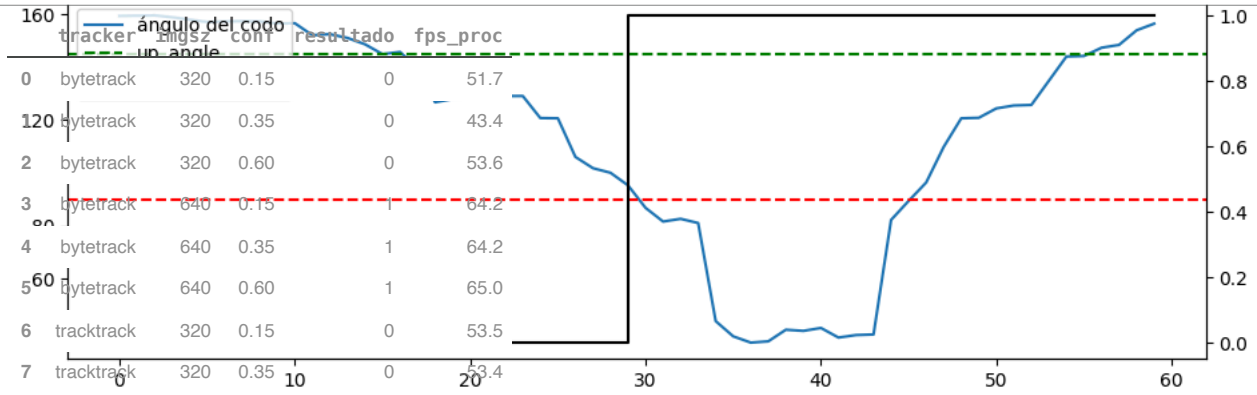
Usa un modelo de **pose** (17 puntos clave COCO). `kpts=[6, 8, 10]` = hombro, codo y muñeca **derechos**: el ángulo en el codo decide si la etapa es `up` o `down`.

Puntos clave: 5/6 hombros · 7/8 codos · 9/10 muñecas · 11/12 caderas · 13/14 rodillas · 15/16 tobillos (impar = izquierdo).

```
gym = solutions.AIGym(model="yolo11n-pose.pt", kpts=[6, 8, 10], up_angle=145.0, down_angle=90.0,
                      conf=0.25, imgsz=640, tracker="bytetrack.yaml", show=False, verbose=False, device=DEVICE)
ext_gym = lambda r: {"reps": r.workout_count[0] if r.workout_count else 0,
                     "angulo": r.workout_angle[0] if r.workout_angle else np.nan,
                     "etapa": r.workout_stage[0] if r.workout_stage else "-"}
df2, seg = procesar_video(gym, VIDEO_POSE, "eq2_gym.mp4", extra=ext_gym)
print("Repeticiones:", df2.reps.max(), "(el video demo dura solo 2 s; graben uno propio de ≤ 10 s para más repeticio
mostrar_video("eq2_gym.mp4")
fig, ax = plt.subplots(figsize=(11, 3.5))
ax.plot(df2.frame, df2.angulo, label="ángulo del codo")
ax.axhline(145, ls="--", c="g", label="up_angle"); ax.axhline(90, ls="--", c="r", label="down_angle")
ax2 = ax.twinx(); ax2.step(df2.frame, df2.reps, c="k", label="reps"); ax.legend(loc="upper left"); plt.show()
```

```
Repeticiones: 1 (el video demo dura solo 2 s; graben uno propio de ≤ 10 s para más repeticiones)
```

```
# 🔧 Barrido obligatorio (+ prueben distintos up_angle/down_angle y kpts=[5, 7, 9])
b2 = barrido_solucion(lambda c, sz, tr: solutions.AIGym(model="yolo11n-pose.pt", kpts=[6, 8, 10], up_angle=145.0,
                      down_angle=90.0, conf=c, imgsz=sz, tracker=tr, show=False, verbose=False, device=DEVICE),
                      VIDEO_POSE, metrica=lambda d: d.reps.max(), extra=ext_gym)
b2
```



	tracker	imgsz	conf	resultado	fps_proc
0	bytetrack	320	0.15	0	51.7
1	bytetrack	320	0.35	0	43.4
2	bytetrack	320	0.60	0	53.6
3	bytetrack	640	0.15	1	64.2
4	bytetrack	640	0.35	1	64.2
5	bytetrack	640	0.60	1	65.0
6	tracktrack	320	0.15	0	53.5
7	tracktrack	320	0.35	0	53.4
8	tracktrack	320	0.60	0	56.0
9	tracktrack	640	0.15	1	54.6
10	tracktrack	640	0.35	1	52.9
11	tracktrack	640	0.60	1	39.4

✍️ **Proyecto 2:** (a) Con la gráfica del ángulo, ¿los umbrales 145°/90° están bien elegidos? Propón los tuyos y justifica. (b) ¿Qué pasa si usas el brazo izquierdo `[5, 7, 9]`? (c) ¿Por qué un `conf` alto puede *bajar* el número de repeticiones? (d) ¿Qué puntos clave usarías para contar sentadillas?

Proyecto 3 — `Heatmap`: mapa de calor de circulación

Acumula la presencia de cada objeto seguido en un mapa; si se le da `region`, también cuenta.

```
heat = solutions.Heatmap(model="yolo11n.pt", colormap=cv2.COLORMAP_JET, classes=[0], conf=0.25, imgsz=640, tracker="
                         show=False, verbose=False, device=DEVICE)
df3, seg = procesar_video(heat, VIDEO_CALLE, "eq3_heatmap.mp4")
mostrar_video("eq3_heatmap.mp4")
mapa = heat.heatmap.copy()                      # matriz acumulada (alto × ancho)
plt.figure(figsize=(9, 5)); plt.imshow(mapa, cmap="jet"); plt.colorbar(label="intensidad acumulada")
plt.title("Mapa de calor acumulado"); plt.axis("off"); plt.show()
# ¿En qué tercio del video (izquierda / centro / derecha) circula más gente?
tercios = np.array_split(mapa, 3, axis=1)
print({n: round(float(t.sum() / mapa.sum()), 2) for n, t in zip(["izq", "centro", "der"], tercios)})
```